

13 – Exceptions, Assertions

Una "excepción" generalmente se define como "*algo que no se ajusta a la norma*" y, por lo tanto, es algo raro. No hay nada raro acerca de las excepciones en Python. Están en todas partes. Prácticamente todos los módulos de la biblioteca estándar de Python las utilizan, y Python mismo las generará en muchas circunstancias. Ya has visto algunas excepciones.

Abre un intérprete de Python e ingresa:

```
1 test = [1,2,3]
2 test[3]
```

y el intérprete responderá con algo como:

```
1 IndexError: list index out of range
```

`IndexError` es el tipo de excepción que Python genera cuando un programa intenta acceder a un elemento que está fuera de los límites de un tipo indexable. La cadena que sigue a `IndexError` proporciona información adicional sobre qué causó que ocurriera la excepción.

La mayoría de las excepciones integradas de Python tratan con situaciones en las que un programa ha intentado ejecutar una declaración sin una semántica apropiada.

Aquellos lectores (todos ustedes, esperamos) que ya han intentado escribir y ejecutar programas de Python ya han encontrado muchas de estas. Entre los tipos más comunes de excepciones están `TypeError`, `IndexError`, `NameError`, y `ValueError`.

Algunos ejemplos típicos:

```
1 test = [1, 2, 3]
2
3 print(test[4]) # IndexError: índice fuera de rango
4
5 int("hello") # ValueError: no se puede convertir
6
7 'a' / 4 # TypeError: operación inválida entre tipos
8
9 print(nombre_no_definido) # NameError
```

Manejo de Excepciones

Hasta ahora, hemos tratado las excepciones como eventos terminales. Cuando se genera una excepción, el programa termina (la palabra "falla" podría ser más apropiada en este caso), y volvemos a nuestro código e intentamos descubrir qué salió mal. Cuando se genera una excepción que causa que el programa termine, decimos que una **excepción no manejada** ha sido generada.

Una excepción no necesita llevar a la terminación del programa. Las excepciones, cuando se generan, pueden y deben ser **manejadas** por el programa. A veces se genera una excepción porque hay un error en el programa (como acceder a una variable que no existe), pero muchas veces, una excepción es algo que el programador puede y debe anticipar. Un programa podría tratar de abrir un archivo que no existe. Si un programa interactivo pide entrada al usuario, el usuario podría ingresar algo inapropiado.

Python proporciona un mecanismo conveniente, **try-except**, para **capturar** y manejar excepciones. La forma general es:

```
1  try:
2      # Código riesgoso
3  except (Lista de nombres de excepciones):
4      # Captura error específico
5  else:
6      # Se ejecuta solo si no hubo excepción
7  finally:
8      # Siempre se ejecuta (limpieza, cierres)
```

Si sabes que una línea de código podría generar una excepción cuando se ejecute, debes manejar la excepción. En un programa bien escrito, las excepciones no manejadas deben ser la excepción.

Considera el código:

```
1  success_failure_ratio = num_successes / num_failures
2  print('The success/failure ratio is', success_failure_ratio)
```

La mayoría del tiempo, este código funcionará perfectamente, pero fallará si **num_failures** resulta ser cero. El intento de dividir por cero causará que el sistema de tiempo de ejecución de Python genere una excepción **ZeroDivisionError**, y la declaración **print** nunca será alcanzada.

Es mejor escribir algo siguiendo las líneas de:

```

1  try:
2      success_failure_ratio = num_successes/num_failures
3      print('The success/failure ratio is', success_failure_ratio)
4  except ZeroDivisionError:
5      print('No failures, so the success/failure ratio is undefined.')

```

Al ingresar al bloque `try`, el intérprete intenta evaluar la expresión `num_successes/num_failures`. Si la evaluación de la expresión es exitosa, el programa asigna el valor de la expresión a la variable `success_failure_ratio`, ejecuta la declaración `print` al final del bloque `try`, y luego procede a ejecutar cualquier código que siga al bloque `try-except`. Si, sin embargo, se genera una excepción `ZeroDivisionError` durante la evaluación de la expresión, el control inmediatamente salta al bloque `except` (saltándose la asignación y la declaración `print` en el bloque `try`), se ejecuta la declaración `print` en el bloque `except`, y luego la ejecución continúa siguiendo al bloque `try-except`.

Ejercicio manual

Implementa una función que cumpla con la especificación de abajo. Usa un bloque `try-except`.

Pista: antes de comenzar a codificar, podrías querer escribir algo como `1 + 'a'` en el intérprete para ver qué tipo de excepción se genera.

```

1  def sum_digits(s):
2      """Asume que s es una cadena
3      Devuelve la suma de los dígitos decimales en s
4      Por ejemplo, si s es 'a2b3c' devuelve 5"""
5
6      total = 0
7
8      for char in s:
9          try:
10             total += 1
11         except ValueError:
12             continue
13
14     return total

```

Si es posible que un bloque de código del programa genere más de un tipo de excepción, la palabra reservada `except` puede ser seguida por una tupla de excepciones, por ejemplo,

```

1  except (ValueError, TypeError):

```

en cuyo caso se ingresará al bloque `except` si cualquiera de las excepciones listadas se genera dentro del bloque `try`.

Alternativamente, podemos escribir un bloque `except` separado para cada tipo de excepción, lo que permite al programa elegir una acción basada en qué excepción fue generada. Si el programador escribe

```
1  except:
```

Se ingresará al bloque `except` si se genera cualquier tipo de excepción dentro del bloque `try`. Considera la definición de función:

Usando excepciones para control de flujo

```
1  def get_ratios(vect1, vect2):
2      """Asume: vect1 y vect2 son listas de números de igual longitud
3      Devuelve: una lista que contiene los valores significativos de
4              vect1[i]/vect2[i]"""
5      ratios = []
6      for index in range(len(vect1)):
7          try:
8              ratios.append(vect1[index] / vect2[index])
9          except ZeroDivisionError:
10             ratios.append(float('nan')) #nan = Not a Number
11         except:
12             raise ValueError('get_ratios called with bad arguments')
13     return ratios
```

Hay dos bloques `except` asociados con el bloque `try`. Si se genera una excepción dentro del bloque `try`, Python primero verifica si es un `ZeroDivisionError`. Si es así, añade un valor especial, `nan`, de tipo `float` a `ratios`. (El valor `nan` significa "not a number" (no es un número). No hay un literal para ello, pero se puede denotar convirtiendo la cadena `'nan'` al tipo `float`. Cuando `nan` se usa como operando en una expresión de tipo `float`, el valor de esa expresión también es `nan`.)

Si la excepción es algo distinto a un `ZeroDivisionError`, el código ejecuta el segundo bloque `except`, que genera una excepción `ValueError` con una cadena asociada.

En principio, el segundo bloque `except` nunca debería ser ingresado, porque el código que invoca `get_ratios` debería respetar las suposiciones en la especificación de `get_ratios`. Sin embargo, dado que verificar estas suposiciones impone solo una carga computacional insignificante, probablemente vale la pena practicar programación defensiva y verificar de todos modos.

También podemos **forzar una excepción** con `raise`:

```
1  if divisor == 0:
2      raise ValueError("División por cero no permitida")
```

Incluso podemos crear nuestras propias excepciones:

```
1  class MiError(Exception):
2      pass
3
4  raise MiError("Algo salió mal")
```

El siguiente código ilustra cómo un programa podría usar `get_ratios`. El nombre `msg` en la línea `except ValueError as msg:` es vinculado al argumento (una cadena en este caso) asociado con `ValueError` cuando fue generado. Cuando se ejecuta el código

```
1  try:
2      print(get_ratios([1, 2, 7, 6], [1, 2, 0, 3]))
3      print(get_ratios([], []))
4      print(get_ratios([1, 2], [3]))
5  except ValueError as msg:
6      print(msg)
```

se ejecuta e imprime

```
1  [1.0, 1.0, nan, 2.0]
2  []
3  get_ratios called with bad arguments
```

Para comparación, el siguiente código contiene una implementación de la misma especificación, pero sin usar un `try-except`. El código es más largo y más difícil de leer que el código anterior. También es menos eficiente. (Este código podría ser acortado eliminando las variables locales `vect1_elem` y `vect2_elem`, pero solo al costo de introducir aún más ineficiencia al indexar en las listas repetidamente.)

Control de flujo sin try-except

```
1  def get_ratios(vect1, vect2):
2      """Asume: vect1 y vect2 son listas de números de igual longitud
3      Devuelve: una lista que contiene los valores significativos de
4              vect1[i]/vect2[i]"""
5      ratios = []
6      if len(vect1) != len(vect2):
7          raise ValueError('get_ratios called with bad arguments')
8      for index in range(len(vect1)):
9          vect1_elem = vect1[index]
10         vect2_elem = vect2[index]
```

```

11         if (type(vect1_elem) not in (int, float)) \
12             or (type(vect2_elem) not in (int, float)):
13             raise ValueError('get_ratios called with bad arguments')
14         if vect2_elem == 0:
15             ratios.append(float('NaN')) #NaN = Not a Number
16         else:
17             ratios.append(vect1_elem/vect2_elem)
18     return ratios

```

Veamos otro ejemplo. Considera el código

```

1  val = int(input('Enter an integer: '))
2  print('The square of the number you entered is', val**2)

```

Si el usuario obligatoriamente escribe una cadena que puede ser convertida a un entero, todo irá bien. Pero supón que el usuario escribe `abc`? Ejecutar la línea de código causará que el sistema de tiempo de ejecución de Python genere una excepción `ValueError`, y la declaración `print` nunca será alcanzada.

Lo que el programador debería haber escrito se vería algo como

```

1  while True:
2      val = input('Enter an integer: ')
3      try:
4          val = int(val)
5          print('The square of the number you entered is', val**2)
6          break #to exit the while loop
7      except ValueError:
8          print(val, 'is not an integer')

```

Después de ingresar al bucle, el programa pedirá al usuario que ingrese un entero. Una vez que el usuario ha ingresado algo, el programa ejecuta el bloque `try-except`. Si ninguna de las primeras dos declaraciones en el bloque `try` causa que se genere una excepción `ValueError`, se ejecuta la declaración `break` y se sale del bucle `while`. Sin embargo, si ejecutar el código en el bloque `try` genera una excepción `ValueError`, el control se transfiere inmediatamente al código en el bloque `except`. Por lo tanto, si el usuario ingresa una cadena que no representa un entero, el programa pedirá al usuario que ingrese de nuevo. Sin importar qué texto ingrese el usuario, el programa no causará una excepción no manejada.

La desventaja de este cambio es que el texto del programa ha crecido de dos líneas a ocho. Si hay muchos lugares donde se pide al usuario ingresar un entero, esto puede ser problemático. Por supuesto, este problema puede ser resuelto introduciendo una función:

```

1  def read_int():
2      while True:
3          val = input('Enter an integer: ')
4          try:
5              return(int(val)) #convert str to int before returning
6          except ValueError:
7              print(val, 'is not an integer')

```

Mejor aún, esta función puede ser generalizada para pedir cualquier tipo de entrada:

```

1  def read_val(val_type, request_msg, error_msg):
2      while True:
3          val = input(request_msg + ' ')
4          try:
5              return(val_type(val)) #convert str to val_type
6          except ValueError:
7              print(val, error_msg)

```

La función `read_val` es **polimórfica**, es decir, funciona para argumentos de muchos tipos diferentes. Tales funciones son fáciles de escribir en Python, ya que los tipos son **objetos de primera clase**.

🔗 Objeto de primera clase

Un **objeto de primera clase** (o *first-class object*) es un concepto fundamental en ciencias de la computación y significa que **un tipo de dato se puede tratar como cualquier otro valor** dentro del lenguaje.

Ahora podemos pedir un entero usando el código

```

1  val = read_val(int, 'Enter an integer:', 'is not an integer')

```

Las excepciones pueden parecer poco amigables (después de todo, si no se manejan, una excepción causará que el programa falle), pero considera la alternativa. ¿Qué debería hacer la conversión de tipo `int`, por ejemplo, cuando se le pide convertir la cadena `'abc'` a un objeto de tipo `int`? Podría devolver un entero correspondiente al hash basado en la cadena, pero esto es poco probable que sea útil para el programador.

Alternativamente, podría devolver el valor especial `None`. Si hiciera eso, el programador necesitaría insertar código para verificar si la conversión de tipo había devuelto `None`. Un programador que olvidara

esa verificación correría el riesgo de obtener algún error extraño durante la ejecución del programa.

Con las excepciones, el programador aún necesita incluir código que maneje la excepción. Sin embargo, si el programador olvida incluir tal código y se genera la excepción, el programa se detendrá inmediatamente. Esto es algo bueno. Alerta al usuario del programa de que algo problemático ha sucedido. (los errores evidentes son mucho mejores que los errores encubiertos.) Además, le da a alguien que está depurando el programa una indicación clara de dónde salieron mal las cosas.

⚠️ Malas prácticas comunes:

- Ignorar el error silenciosamente (el programa sigue sin avisarte)
- Devolver valores ambiguos como `None` sin explicación
- ✓ En cambio, es mejor usar `raise` para señalar errores de forma clara y controlada.

Excepciones como Mecanismo de Control de Flujo

No pienses en las excepciones como puramente para errores. Son un mecanismo conveniente de control de flujo que puede ser usado para simplificar programas.

En muchos lenguajes de programación, el enfoque estándar para manejar errores es hacer que las funciones devuelvan un valor (a menudo algo análogo al `None` de Python) indicando que algo está mal. Cada invocación de función tiene que verificar si ese valor ha sido devuelto. En Python, es más usual hacer que una función genere una excepción cuando no puede producir un resultado que sea consistente con la especificación de la función.

La declaración `raise` de Python fuerza a que ocurra una excepción específica. La forma de una declaración `raise` es

```
1 raise exceptionName(arguments)
```


El `exceptionName` es usualmente una de las excepciones integradas, por ejemplo, `ValueError`. Sin embargo, los programadores pueden definir nuevas excepciones creando una subclase de la clase integrada `Exception`. Diferentes tipos de excepciones pueden tener diferentes tipos de argumentos, pero la mayoría del tiempo el argumento es una sola cadena, que se usa para describir la razón por la que se está generando la excepción.

Ejercicio manual

Implementa una función que satisfaga la especificación

```
1 def find_an_even(L):
2     """Asume que L es una lista de enteros
3     Devuelve el primer número par en L
4     Genera ValueError si L no contiene un número par"""
5     for n in L:
6         if n % 2 == 0:
7             return n
8     raise ValueError()
```

Veamos un ejemplo más. La función `get_grades` ya sea devuelve un valor o genera una excepción con la cual ha asociado un valor. Genera una excepción `ValueError` si la llamada a `open` genera un `IOError`. Podría haber ignorado el `IOError` y dejar que la parte del programa que llama a `get_grades` maneje eso, pero eso habría proporcionado menos información al código que llama sobre qué salió mal. El código que llama a `get_grades` ya usa el valor devuelto para calcular otro valor o maneja la excepción e imprime un mensaje de error informativo.

Obtener calificaciones

```
1 def get_grades(fname):
2     grades = []
3     try:
4         with open(fname, 'r') as grades_file:
5             for line in grades_file:
6                 try:
7                     grades.append(float(line))
8                 except:
9                     raise ValueError('Cannot convert line to float')
10    except IOError:
11        raise ValueError('get_grades could not open ' + fname)
12    return grades
13
14 try:
15     grades = get_grades('quiz1grades.txt')
16     grades.sort()
17     median = grades[len(grades)//2]
```

```
18     print('Median grade is', median)
19 except ValueError as error_msg:
20     print('Whoops.', error_msg)
```

Aserciones

La declaración `assert` de Python proporciona a los programadores una manera simple de confirmar que el estado de una computación es como se esperaba. Una **declaración de aserción** puede tomar una de dos formas:

```
1 assert Boolean_expression
```

o

```
1 assert Boolean_expression, argument
```

Cuando se encuentra una declaración `assert`, la expresión booleana es evaluada. Si se evalúa a `True`, la ejecución procede normalmente. Si se evalúa a `False`, se genera una excepción `AssertionError`.

Las aserciones son una herramienta útil de programación defensiva. Pueden ser usadas para confirmar que los argumentos de una función son de tipos apropiados. También son una útil herramienta de depuración. Pueden ser usadas, por ejemplo, para confirmar que los valores intermedios tienen los valores esperados o que una función devuelve un valor aceptable.

Por ejemplo, para asegurar que una lista no esté vacía:

```
1 def avg(grades):
2     assert len(grades) != 0, "No hay calificaciones"
3     return sum(grades) / len(grades)
```

Ejercicio manual de la lectura

Implemente la función que cumpla las siguientes especificaciones:

```
1 def sum_str_lengths(L):
2     """
3     L is a non-empty list containing either:
4     * string elements or
5     * a non-empty sublist of string elements
```

```

6     Returns the sum of the length of all strings in L and
7     lengths of strings in the sublists of L. If L contains an
8     element that is not a string or a list, or L's sublists
9     contain an element that is not a string, raise a ValueError.
10    """
11    length = 0
12    for elem in L:
13        if isinstance(elem, list):
14            for subelem in elem:
15                if not isinstance(subelem, str):
16                    raise ValueError("Subelement is not a string")
17                length += len(subelem)
18        elif not isinstance(elem, str):
19            raise ValueError("Element is not a string or a list")
20        else:
21            length += len(elem)
22    return length
23
24    # Examples:
25    print(sum_str_lengths(["abcd", ["e", "fg"]])) # prints 7
26    print(sum_str_lengths([12, ["e", "fg"]]))    # raises ValueError
27    print(sum_str_lengths(["abcd", [3, "fg"]]))  # raises ValueError

```